



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Magistrale in Informatica

REPORT PROGETTO

Software Dependability

DOCENTE

Prof. Dario Di Nucci

Università degli Studi di Salerno

STUDENTE

Francesco Maria Torino

0522501879

Anno Accademico 2025-2026

Indice

Elenco delle Figure	iii
Elenco delle Tabelle	v
1 Introduzione	1
1.1 Contesto e Motivazione	1
1.2 Motivazione all'Integrazione della Dependability	4
1.3 Obiettivi della Dependability	5
2 Applicazione dei Principi di Dependability a PlayWise	7
2.1 OpenJML – Verifica Formale	7
2.2 JaCoCo – Evoluzione della Copertura dei Test	9
2.3 Codecov – Analisi dei Tempi e Copertura dei Test	13
2.4 PITest – Mutation Testing	14
2.5 JMH – Microbenchmarking e Ottimizzazione delle Prestazioni	16
2.5.1 Risultati pre-ottimizzazione	18
2.5.2 Ottimizzazione del cost factor di BCrypt	18
2.5.3 Risultati post-ottimizzazione	19
2.6 Docker – Deployment e Ripetibilità	21
2.7 CI/CD e Analisi di Sicurezza	22
2.7.1 GitGuardian – Secret Detection	23

2.7.2	Snyk – Vulnerability Scanning	24
2.7.3	SonarCloud – Static Code Analysis	25
3	Conclusioni	27
3.1	Risultati Principali	27
3.2	Limitazioni	29
3.3	Sviluppi Futuri	29
3.4	Collaboratori	30

Elenco delle figure

2.1	Verifica formale con ESC eseguita con successo sul modulo <i>GeneticEngine</i> .	8
2.2	Runtime Assertion Checking (RAC) completato con esito positivo.	8
2.3	Report iniziale JaCoCo – Copertura parziale (85% line coverage, 80% branch coverage).	10
2.4	Report finale JaCoCo – Copertura completa (100% line coverage, 100% branch coverage).	11
2.5	Tabella dei tempi medi dei test esportata da Codecov.	13
2.6	Report iniziale PITest – presenza di mutanti sopravvissuti.	14
2.7	Report finale PITest – raggiunto 100% di Mutation Coverage e Test Strength.	14
2.8	Esecuzione dei benchmark JMH sui metodi analizzati	17
2.9	Esecuzione automatica del deploy Docker al termine della pipeline CI/CD.	21
2.10	Pipeline CI/CD su GitHub Actions: esecuzione automatizzata del workflow <code>build.yml</code> .	23
2.11	Report finale GitGuardian: tutte le credenziali esposte revocate e pipeline conforme.	24
2.12	Report iniziale Snyk: vulnerabilità rilevate nelle dipendenze e configurazioni.	25

2.13	Report finale Snyk: tutte le vulnerabilità risolte e librerie aggiornate.	25
2.14	Report iniziale SonarCloud: code smell e hotspot di sicurezza individuati.	26
2.15	Report finale SonarCloud: codice conforme alle metriche di qualità e sicurezza.	26

Elenco delle tabelle

2.1	Test più lenti (<i>slowest tests</i>) rilevati da Codecov	16
2.2	Risultati JMH (fase pre-ottimizzazione) — metrica: AverageTime . .	18
2.3	Risultati JMH (fase post-ottimizzazione) — metrica: AverageTime . .	19

CAPITOLO 1

Introduzione

1.1 Contesto e Motivazione

La piattaforma **PlayWise** nasce come progetto accademico sviluppato nell'ambito dei corsi di *Software Engineering for AI* e *Basi di Dati 2* presso l'Università degli Studi di Salerno, con l'obiettivo di progettare un sistema completo, moderno e affidabile per la gestione e l'analisi delle recensioni videoludiche. Ispirata a portali internazionali come *Metacritic* e *OpenCritic*, la piattaforma consente agli utenti di esplorare videogiochi, pubblicare recensioni, confrontare valutazioni e ricevere raccomandazioni personalizzate, integrando componenti di **software dependability**.

L'architettura originaria del progetto è stata concepita come **microservizio distribuito**, con servizi dedicati alla gestione di utenti, giochi, recensioni e raccomandazioni, ciascuno dotato di un proprio database e comunicazione REST. Tuttavia, ai fini del presente lavoro, tale struttura è stata ricondotta a una **versione monolitica integrata**, al fine di semplificare il processo di sviluppo, testing e integrazione continua (CI/CD), mantenendo comunque i principi di modularità e separazione logica dei componenti.

L'applicazione segue un'architettura a tre livelli:

- **Frontend:** sviluppato in *React*, fornisce un'interfaccia utente interattiva, accessibile e responsiva, conforme alle linee guida WCAG 2.1;
- **Backend:** realizzato in *Spring Boot 3*, gestisce la logica applicativa e i servizi REST, occupandosi della sicurezza, della validazione dei dati e dell'interazione con il database;
- **Database:** implementato con *MongoDB*, archivia in formato documentale tutte le informazioni relative a giochi, utenti e recensioni, sfruttando la flessibilità del modello NoSQL per rappresentare entità eterogenee.

Sul piano funzionale, il sistema permette di:

- visualizzare e filtrare videogiochi per genere, piattaforma e valutazione;
- gestire la pubblicazione e la modifica di recensioni utente;
- amministrare statistiche aggregate (es. distribuzione dei punteggi e top piattaforme);

All'interno della piattaforma **PlayWise**, è stato implementato un **classificatore automatico PEGI** (Pan European Game Information) con l'obiettivo di stimare in modo intelligente la fascia d'età consigliata per ciascun videogioco, basandosi sull'analisi semantica dei contenuti testuali delle descrizioni dei giochi. Il sistema assegna automaticamente un livello di classificazione tra *PEGI 3*, *PEGI 7*, *PEGI 12*, *PEGI 16* e *PEGI 18*, in funzione della presenza di termini e contesti riconducibili a violenza, paura, linguaggio volgare, riferimenti sessuali o dipendenze.

Il modulo di classificazione si basa su un **algoritmo genetico**, progettato per ottimizzare dinamicamente i pesi associati alle diverse categorie di contenuto. Attraverso un processo di evoluzione iterativa, il sistema ricerca la combinazione di parametri che massimizza l'accuratezza di previsione rispetto a un insieme di dati etichettati. Questo approccio consente di ottenere un classificatore adattivo, in grado di affinare progressivamente la propria capacità discriminante anche in presenza di dataset parzialmente rumorosi o sbilanciati.

Dal punto di vista architetturale, il *PEGI Feature Extractor* si occupa della tokenizzazione e dell'analisi linguistica del testo, mentre il modulo *Genetic Engine* gestisce la

generazione e valutazione delle soluzioni candidate. Il risultato finale è un sistema leggero, interpretabile e formalmente verificabile, integrato nel backend *Spring Boot* e validato attraverso tecniche di testing e verifica formale sviluppate nell'ambito del progetto.

L'intero progetto è stato sviluppato con una forte attenzione alla **qualità del software** e alla **manutenibilità del codice**. Sono stati introdotti strumenti di analisi e controllo automatico che hanno reso possibile monitorare e migliorare le metriche di affidabilità e sicurezza:

- **JaCoCo** e **Codecov** per la misurazione della copertura dei test;
- **PITest** per la valutazione dell'efficacia della suite di test tramite mutation testing;
- **OpenJML** per la verifica formale delle pre- e post-condizioni dei metodi critici;
- **JMH** per l'analisi delle prestazioni e l'ottimizzazione dei metodi a maggiore impatto computazionale;
- **Snyk**, **SonarCloud** e **GitGuardian** per il controllo continuo di vulnerabilità, code smell e segreti esposti;
- **Docker** per la containerizzazione e la ripetibilità degli ambienti di esecuzione.

La pipeline CI/CD, realizzata con **GitHub Actions**, integra tutte queste attività in un flusso automatizzato che comprende build, test, analisi statica, mutation testing, verifica delle soglie di qualità e distribuzione automatica su DockerHub. Questo approccio ha permesso di creare un ecosistema di sviluppo altamente riproducibile, dove ogni commit produce risultati misurabili in termini di copertura, mutazioni uccise, vulnerabilità risolte e performance migliorate.

PlayWise rappresenta quindi non solo un'applicazione web completa, ma anche un banco di prova concreto per l'applicazione dei principi di *Software Dependability*. Attraverso l'integrazione di pratiche di testing avanzato, sicurezza automatizzata e verifica formale, il progetto dimostra come sia possibile conciliare la ricerca accademica con l'ingegneria del software industriale, creando un sistema robusto, scalabile e sostenibile nel tempo.

Il progetto, inizialmente focalizzato sulla gestione e visualizzazione delle recensioni, è stato successivamente esteso con un modulo dedicato alla **classificazione automatica del livello PEGI** dei videogiochi, basato sull’analisi semantica della descrizione testuale. Tale estensione, pur non configurandosi come un sistema di intelligenza artificiale completo, introduce un componente algoritmico che consente di sperimentare tecniche di valutazione automatica e di verifica formale della correttezza del software.

1.2 Motivazione all’Integrazione della Dependability

Nell’ambito dell’ingegneria del software moderna, la **dependability** rappresenta l’insieme delle qualità che rendono un sistema affidabile, sicuro e disponibile. Secondo la definizione del gruppo IFIP 10.4, essa può essere intesa come la *“trustworthiness of a computing system which allows justified reliance on the service it delivers”*. In altre parole, un sistema è dependable se garantisce che i propri servizi siano erogati correttamente e senza interruzioni indesiderate, riducendo al minimo la probabilità di guasti, vulnerabilità o comportamenti anomali.

Nel contesto di **PlayWise**, l’introduzione della dipendenza dal paradigma della dependability ha permesso di:

- prevenire errori di implementazione e di logica applicativa mediante **verifica formale in JML** (Java Modeling Language) con **OpenJML**, che assicura la correttezza dei metodi principali, tra cui l’algoritmo genetico di classificazione PEGI;
- valutare la robustezza del codice tramite **JaCoCo** per l’analisi della code coverage e **PITest** per il mutation testing, garantendo un’elevata qualità e varietà dei test unitari;
- misurare i tempi medi di esecuzione dei test e individuare i metodi critici tramite l’integrazione di **Codecov**, utilizzato per l’analisi quantitativa delle durate e per la selezione delle funzioni da ottimizzare mediante **microbenchmark JMH**;

- ottimizzare le prestazioni dei componenti più esigenti con **JMH**, attraverso benchmark mirati ai metodi più onerosi emersi dall'analisi di Codecov;
- integrare meccanismi di **security-by-design** nel ciclo CI/CD, attraverso analisi automatiche condotte da **GitGuardian**, **Snyk** e **SonarCloud**, per individuare vulnerabilità, dipendenze a rischio e segreti esposti;
- assicurare la ripetibilità, la disponibilità e la scalabilità del sistema tramite l'uso di **Docker**, con immagini dell'applicazione pronte all'orchestrazione e pubblicate su DockerHub.

1.3 Obiettivi della Dependability

L'integrazione di tali pratiche ha l'obiettivo di garantire che PlayWise soddisfi i requisiti fondamentali di affidabilità, disponibilità, sicurezza e manutenibilità. In particolare:

- la **reliability** è assicurata da un'ampia suite di test, dall'uso combinato di JaCoCo e PITest e dalla verifica formale dei metodi critici tramite OpenJML;
- la **availability** è supportata dalla containerizzazione, dalla presenza di immagini Docker ufficiali e dal monitoraggio continuo in pipeline CI/CD;
- la **security** è garantita da strumenti di analisi statica e dinamica automatizzata come GitGuardian, Snyk e SonarCloud, integrati nel processo di build;
- la **performance reliability** è rafforzata dall'uso di Codecov e JMH, che permettono di identificare colli di bottiglia e validare sperimentalmente l'efficacia delle ottimizzazioni introdotte;
- la **maintainability** è sostenuta dall'organizzazione modulare del codice, dall'adozione di metriche di qualità e dalla tracciabilità delle dipendenze nel ciclo di vita del software.

In sintesi, la combinazione di verifica formale, testing avanzato, analisi delle prestazioni e sicurezza automatizzata consente a PlayWise di evolvere da un semplice

progetto universitario a una piattaforma *dependable*, capace di fornire servizi affidabili, resilienti e verificabili. Il risultato finale è un sistema che unisce la concretezza di una web application reale con le metodologie di **Software Dependability**, offrendo un ambiente sperimentale ideale per lo studio e la pratica della qualità del software.

Applicazione dei Principi di Dependability a PlayWise

2.1 OpenJML – Verifica Formale

OpenJML è stato impiegato per la verifica formale dei metodi chiave del sistema, con l'obiettivo di garantire la correttezza funzionale e la coerenza logica delle operazioni più sensibili. L'analisi è stata condotta in modo approfondito sul modulo *GeneticEngine*, che rappresenta il cuore computazionale del sistema di classificazione PEGI. Tale modulo, essendo completamente autonomo e privo di dipendenze da *Spring Boot* o da framework web, si presta in modo ideale alla verifica formale completa tramite specifiche JML.

Queste specifiche hanno permesso di assicurare la correttezza del processo evolutivo, la coerenza dimensionale della popolazione, la stabilità post-mutazione e la positività del valore restituito dalla funzione di fitness, garantendo l'assenza di errori aritmetici e comportamentali.

La verifica è stata condotta in due modalità complementari. La prima, denominata **ESC (Extended Static Checking)**, ha permesso di eseguire un controllo statico delle specifiche JML e delle condizioni di correttezza logica a compile-time, mediante il comando `openjml -esc`. La seconda, nota come **RAC (Runtime Assertion Checking)**, ha invece comportato la strumentazione dinamica del bytecode tramite

`openjml -rac`, verificando a runtime la validità dei contratti in fase di esecuzione. Entrambe le analisi si sono concluse con esito positivo, come mostrato nelle Figure 2.1 e 2.2, confermando la conformità del codice alle specifiche formali e l'assenza di violazioni di contratto.

```
francescomariotino@MacBook-Pro-di-Francesco games-project % openjml -esc -timeout=10 \
-classpath "$(mvn -q dependency:build-classpath -Dmdep.outputAbsoluteArtifactFilename=true \
-Dmdep.outputFile=/dev/stdout -DincludeScope=compile):target/classes" \
-sourcepath src/main/java \
src/main/java/com/games/games_project/geneticalgorithm/*.java
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::staticFieldBase has been called by com.google.inject.internal.aop.HiddenClassDefiner (file:/opt/homebrew/Cellar/maven/3.9.11/libexec/lib/guice-5.1.0-classic.jar)
WARNING: Please consider reporting this to the maintainers of class com.google.inject.internal.aop.HiddenClassDefiner
WARNING: sun.misc.Unsafe::staticFieldBase will be removed in a future release
```

Figura 2.1: Verifica formale con ESC eseguita con successo sul modulo *GeneticEngine*.

```
WARNING: Please consider reporting this to the maintainers of class com.google.inject.internal.aop.HiddenClassDefiner
WARNING: sun.misc.Unsafe::staticFieldBase will be removed in a future release
=== Test 1: estrazione feature ===
Ratio = 0.2

=== Test 2: testo violento ===
Ratio = 5.0

=== Test 3: errore intenzionale ===
Violazione JML catturata (estrazione): java.lang.NullPointerException: Cannot invoke "String.toLowerCase(java.util.Locale)" because "text" is null

=== Test 4: GeneticEngine - evoluzione ===
Soluzione evoluta:
w[0] = 0,0663
w[1] = 0,1477
w[2] = 0,1827
w[3] = 0,0153
w[4] = -0,0463
w[5] = 0,1393
w[6] = 0,2301
```

Figura 2.2: Runtime Assertion Checking (RAC) completato con esito positivo.

La verifica è stata limitata al modulo genetico poiché i componenti basati su *Spring Boot* e le API REST utilizzano riflessione e meccanismi di iniezione delle dipendenze che interferiscono con il processo di analisi statica di OpenJML. La generazione dinamica di classi e la gestione automatica del contesto applicativo impediscono infatti la risoluzione dei riferimenti a compile-time. Per garantire risultati affidabili e significativi, è stato dunque scelto di concentrare la verifica sul core logico del sistema, dove la formalizzazione JML risulta più rigorosa ed efficace.

2.2 JaCoCo – Evoluzione della Copertura dei Test

Per la valutazione della copertura del codice è stato scelto di concentrare l'attenzione sui moduli più significativi del sistema, ovvero **Controller**, **Service**, **Algoritmo Genetico** e **Utility**. Questi moduli rappresentano le componenti chiave dell'applicazione: i Controller gestiscono la comunicazione REST e la validazione dei parametri, i Service contengono la logica di business e le eccezioni, il modulo genetico incapsula gli algoritmi di classificazione PEGI, mentre le Utility forniscono funzioni trasversali di supporto. La scelta di testare prioritariamente queste aree deriva dal loro impatto diretto sulla **correttezza funzionale**, sull'**affidabilità** e sulla **manutenibilità** del sistema.

La generazione dei test è stata automatizzata con l'ausilio di **ChatGPT**, impiegato per produrre casi di test *JUnit 5* completi e sistematici. A tal fine è stato utilizzato il prompt riportato nel paragrafo *Prompt utilizzato per la generazione dei test (ChatGPT)*, basato sulle tecniche di *Category Partitioning* ed *Edge/Boundary Analysis*, che ha permesso di esplorare in modo esaustivo le combinazioni di input e i percorsi logici del codice.

L'analisi della copertura del codice è stata condotta tramite **JaCoCo**, integrato nel ciclo di build *Maven* e successivamente nella pipeline **CI/CD** di GitHub Actions. Lo strumento ha permesso di valutare in modo quantitativo l'efficacia della suite di test, misurando la percentuale di linee, rami, metodi e istruzioni effettivamente eseguite durante i test unitari.

In una prima fase, i risultati di copertura mostravano valori medi pari all'**85% di line coverage** e all'**80% di branch coverage**, evidenziando porzioni di codice non ancora testate in modo esaustivo, in particolare all'interno dei package *Service* e *Controller*. L'analisi dei report HTML generati da JaCoCo (Figura 2.3) ha consentito di individuare le aree meno coperte, tra cui i metodi relativi alla gestione delle eccezioni, ai flussi condizionali multipli e alle operazioni di validazione dei dati in ingresso.

games-project

games-project

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.games.games_project.service.impl		72%		68%	27	71	74	270	10	30	0	4
com.games.games_project.controller		97%		100%	1	29	2	60	1	23	0	5
com.games.games_project.geneticalgorithm		99%		94%	3	47	0	106	0	21	0	6
com.games.games_project.utils		98%		87%	2	15	0	60	0	7	0	2
Total	358 of 2.457	85%	31 of 162	80%	33	162	76	496	11	81	0	17

Figura 2.3: Report iniziale JaCoCo – Copertura parziale (85% line coverage, 80% branch coverage).

Per incrementare la copertura, la suite di test è stata migliorata iterativamente con l’ausilio di **ChatGPT**, utilizzato come assistente automatico nella generazione di casi di test *JUnit 5*. Attraverso prompt strutturati basati sulle tecniche di *Category Partitioning* ed *Edge/Boundary Analysis*, sono stati prodotti test parametrizzati in grado di esplorare sistematicamente:

- valori limite e input non validi;
- condizioni di errore e rami alternativi;
- comportamenti rari e percorsi eccezionali nei metodi chiave.

I test generati automaticamente sono poi stati raffinati manualmente per adattarli al contesto specifico dei moduli principali del progetto — *Controller*, *Service*, *Algoritmo Genetico* e *Utility* — assicurando che ciascun caso coprisse percorsi logici effettivamente rilevanti per l’esecuzione. Le nuove versioni dei test sono state integrate nel flusso **CI/CD**, permettendo di monitorare in modo continuo l’evoluzione della copertura a ogni build.

Al termine del processo iterativo, i risultati mostrati in Figura 2.4 confermano il raggiungimento della **copertura totale: 100% di line coverage e 100% di branch coverage** su tutti i package analizzati. La copertura completa ha garantito una validazione strutturale esaustiva del sistema, riducendo la probabilità di fault non rilevati e migliorando la confidenza nella qualità del software.

games-project

games-project

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.games.games_project.service.impl		100%		100%	0 71	0 270	0 30	0 4
com.games.games_project.geneticalgorithm		100%		100%	0 47	0 106	0 21	0 6
com.games.games_project.controller		100%		100%	0 29	0 60	0 23	0 5
com.games.games_project.utils		100%		100%	0 15	0 60	0 7	0 2
Total	0 of 2.457	100%	0 of 162	100%	0 162	0 496	0 81	0 17

Figura 2.4: Report finale JaCoCo – Copertura completa (100% line coverage, 100% branch coverage).

L'intero processo è stato eseguito automaticamente all'interno della pipeline CI/CD, che, ad ogni commit o pull request, esegue i test e genera i report di copertura JaCoCo in formato XML e HTML, archiviandoli come artefatti del workflow. Questo ha permesso di mantenere nel tempo la qualità dei test e di garantire che ogni nuova modifica al codice rispettasse la copertura già raggiunta.

L'unione tra tecniche sistematiche di progettazione dei test e strumenti di automazione avanzata ha permesso di consolidare la qualità ingegneristica del progetto, portando la suite di test a un livello di **completezza, affidabilità e ripetibilità** in linea con i principi della *Software Dependability*.

Prompt utilizzato per la generazione dei test (ChatGPT): "Sei un esperto di testing Java. Genera per ogni classe che ti passerò:

1. una tabella di *Category Partitioning*;
2. i file JUnit 5 completi (classi di test) senza commenti, che portino JaCoCo al **100% line** e **100% branch** coverage.

Usa *Category Partitioning* ed edge/boundary analysis per derivare i test. Usa AssertJ per le asserzioni e Mockito quando serve. I test devono essere parametrizzati dove opportuno.

Stile e librerie:

- JUnit 5, Mockito, AssertJ
- Nessun commento nei file di test
- Non introdurre dipendenze esterne non necessarie

- Non inventare metodi: usa solo API pubbliche presenti nelle classi indicate
- Se alcune dipendenze vanno stubbate, usa Mockito

Formato di OUTPUT obbligatorio:

1. Sezione “Category Partitioning” (tabella con Categorie, Partizioni, Valori di confine, Eccezioni attese)
2. Sezione “Test” con **solo** blocchi di codice Java completi:
 - Uno per ogni classe di test necessaria
 - Package corretto `package <PACKAGE_NAME>;`
 - Import JUnit/AssertJ/Mockito
 - Dati di test minimi riproducibili
 - Parametrici per coprire bordi: 0, 1, $0-\varepsilon$ (non valido), $1+\varepsilon$ (non valido), `size=1, size=grande, elitism on/off`
 - Asserzioni di valore (non solo non-null)
 - Dove servono eccezioni: `assertThatThrownBy(...)` con tipo e messaggio

Vincoli di copertura e qualità:

- Ogni ramo `if/else`, `switch`, *early return* e blocco di eccezione deve essere coperto
- Ogni guardia deve lanciare l’eccezione prevista
- Le mutazioni semplici ($\pm$, $/\div$, $>/<$, invert boolean) devono essere uccise dai test

”

2.3 Codecov – Analisi dei Tempi e Copertura dei Test

Il servizio **Codecov** è stato integrato nella pipeline CI/CD per raccogliere automaticamente i report di copertura JaCoCo e analizzare i tempi medi di esecuzione dei test. La sezione dedicata ai test ha permesso di esportare metriche come la durata media e complessiva per ogni metodo testato, consentendo di identificare i punti più onerosi in termini computazionali. Sono stati presi in esame i test che superavano il 95° percentile della durata media dei test, poiché indicativi di possibili inefficienze o di logiche più pesanti da elaborare.

Tra i test più lenti figurano le operazioni di autenticazione, registrazione e consultazione delle recensioni, che sono state successivamente sottoposte ad analisi prestazionali con JMH.

Tests (160) Search by name

Test name	Avg. duration	Time spent	Failure rate	Fake rate	Commits failed	Last run
com.games.games_project.service.impl.GameServiceImplTest:testGetGameDetailsById_SplinterCell_WithMoreThan5Reviews	1.745s	45.362s	0%	0%	0	38 minutes ago
com.games.games_project.service.impl.UserServiceImplTest:testUpdateUser_AllFieldsNull	0.498s	6.467s	0%	0%	0	38 minutes ago
com.games.games_project.service.impl.UserServiceImplTest:testUpdateUser_AllFieldsFilled	0.448s	11.451s	0%	0%	0	38 minutes ago
com.games.games_project.service.impl.UserServiceImplTest:testUpdateUser_AllFieldsEmpty	0.344s	5.853s	0%	0%	0	38 minutes ago
com.games.games_project.service.impl.UserServiceImplTest:testUpdateUser_MixedConditions	0.338s	8.588s	0%	0%	0	38 minutes ago
com.games.games_project.service.impl.UserServiceImplTest:testUpdateUser_AllFieldsNullOrEmpty	0.253s	2.277s	0%	0%	0	4 days ago
com.games.games_project.service.impl.UserServiceImplTest:testRegister_Success	0.249s	6.477s	0%	0%	0	38 minutes ago
com.games.games_project.service.impl.UserServiceImplTest:testLogin_Success	0.237s	6.161s	0%	0%	0	38 minutes ago
com.games.games_project.controller.impl.ReviewControllerTest:testGetReviewsForGame_VerifySortDirection	0.197s	5.128s	0%	0%	0	38 minutes ago
com.games.games_project.service.impl.UserServiceImplTest:testLogin_WrongPassword	0.175s	4.554s	0%	0%	0	38 minutes ago

Figura 2.5: Tabella dei tempi medi dei test esportata da Codecov.

L'utilizzo di Codecov ha quindi permesso di unire l'analisi di copertura a quella temporale, fornendo una visione globale dell'efficienza e dell'efficacia dei test.

2.4 PITest – Mutation Testing

Il framework **PITest** è stato utilizzato per misurare la robustezza della suite di test tramite la **mutation analysis**. Il tool genera mutazioni controllate del codice, verificando se i test esistenti siano in grado di rilevare i comportamenti alterati. Un’alta percentuale di mutazioni “uccise” è indicativa di test efficaci e di un’elevata affidabilità del sistema.

Le classi analizzate sono le stesse già testate con JaCoCo: Controller, Service, Algoritmo Genetico e Utility. L’obiettivo era quello di raggiungere il 100% di Mutation Coverage e Test Strength. Dopo le prime esecuzioni, alcuni mutanti risultavano sopravvissuti; per questo motivo è stato avviato un processo iterativo di miglioramento, in cui i risultati venivano forniti a ChatGPT con la richiesta: *“Questi mutanti non sono stati uccisi. Quali test devo aggiungere per raggiungere l’obiettivo?”*. Sulla base dei suggerimenti generati, sono stati aggiunti nuovi casi di test, fino a ottenere una copertura totale, come mostrato nelle Figure 2.6 e 2.7.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
16	85% 419/495	74% 270/366	87% 270/310

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
com.games.games_project.controller	5	97% 58/60	93% 27/29	96% 27/28
com.games.games_project.geneticalgorithm	5	100% 105/105	79% 84/107	81% 84/104
com.games.games_project.service.impl	4	73% 196/270	62% 113/182	87% 113/130
com.games.games_project.utils	2	100% 60/60	96% 46/48	96% 46/48

Figura 2.6: Report iniziale PITest – presenza di mutanti sopravvissuti.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
16	100% 495/495	100% 366/366	100% 366/366

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
com.games.games_project.controller	5	100% 60/60	100% 29/29	100% 29/29
com.games.games_project.geneticalgorithm	5	100% 105/105	100% 107/107	100% 107/107
com.games.games_project.service.impl	4	100% 270/270	100% 182/182	100% 182/182
com.games.games_project.utils	2	100% 60/60	100% 48/48	100% 48/48

Figura 2.7: Report finale PITest – raggiunto 100% di Mutation Coverage e Test Strength.

Questo approccio iterativo ha permesso di eliminare ogni mutante sopravvissuto, garantendo una suite di test estremamente solida e rafforzando la **reliability** del sistema.

2.5 JMH – Microbenchmarking e Ottimizzazione delle Prestazioni

L'analisi delle prestazioni è iniziata considerando gli **slowest tests** individuati da **Codecov**, ovvero i casi di test che superano il **95° percentile** dei tempi di esecuzione. Questi elementi hanno permesso di identificare in modo sistematico i metodi potenzialmente più onerosi, concentrando l'attenzione sui componenti `service.impl` e `controller`, in particolare sulle operazioni di autenticazione, registrazione e aggiornamento utente.

Le misurazioni sono state eseguite su un **MacBook Pro M3 Pro**, dotato di CPU 12-core e 18 GB di memoria unificata, in grado di garantire stabilità e riproducibilità durante l'esecuzione dei microbenchmark.

La Tabella 2.1 riporta i test oltre il 95° percentile, da cui emerge la necessità di misurazioni più accurate tramite JMH.

Metodo	Tempo medio (s)
testGetGameDetailsById_SplinterCell_WithMoreThan5Reviews	1.742
testUpdateUser_AllFieldsFilled	0.435
testUpdateUser_AllFieldsNull	0.495
testUpdateUser_MixedConditions	0.332
testRegister_Success	0.250
testLogin_Success	0.233
testUpdateUser_AllFieldsEmpty	0.341
testGetReviewsForGame_VerifySortDirection	0.197

Tabella 2.1: Test più lenti (*slowest tests*) rilevati da Codecov

Per ottenere una misurazione accurata e riproducibile sono stati progettati microbenchmark con la libreria **JMH**, adottando una configurazione comune a tutte le classi:

- `@Fork(2)` per eseguire il benchmark in due JVM isolate;
- `@Warmup(iterations = 2)` per stabilizzare le ottimizzazioni JIT;
- `@Measurement(iterations = 3)` per la raccolta delle misure finali;
- `@Threads(8)` per simulare carichi concorrenti realistici su Apple Silicon;
- `@OutputTimeUnit(TimeUnit.MILLISECONDS)`.

I benchmark (`GameServiceBenchmark`, `ReviewControllerBenchmark`, `UserServiceBenchmark`) replicano fedelmente i flussi applicativi, isolando però il costo computazionale puro tramite repository *mockati* con **Mockito**.

modos can be very significant. Please make sure you use the consistent blacknote mode for comparis

Benchmark	Mode	Cnt	Score	Error	Units
GameRepositoryBenchmark.benchmarkCountGamesByPlatform	avgt	3	1,134 ±	3,631	ms/op
GameRepositoryBenchmark.benchmarkFindAll	avgt	3	1,061 ±	3,143	ms/op
GameRepositoryBenchmark.benchmarkFindSuggestions	avgt	3	1,036 ±	3,713	ms/op
GameServiceBenchmark.benchmarkFindSuggestion	avgt	3	1,057 ±	3,249	ms/op
GameServiceBenchmark.benchmarkFindSuggestionEmpty	avgt	3	≈ 10 ⁻⁶		ms/op
GameServiceBenchmark.benchmarkGetGameCountByPlatform	avgt	3	1,137 ±	3,668	ms/op
GameServiceBenchmark.benchmarkGetGameDetailsById	avgt	3	2,692 ±	12,101	ms/op
GameServiceBenchmark.benchmarkGetGamesPaginated	avgt	3	1,859 ±	7,637	ms/op
ReviewRepositoryBenchmark.benchmarkCountReviewsPerMonth	avgt	3	1,187 ±	4,587	ms/op
ReviewRepositoryBenchmark.benchmarkDeleteByAuthor	avgt	3	0,891 ±	2,470	ms/op
ReviewRepositoryBenchmark.benchmarkFindByAuthor	avgt	3	1,288 ±	8,406	ms/op
ReviewRepositoryBenchmark.benchmarkFindByGameId	avgt	3	1,130 ±	4,621	ms/op
ReviewRepositoryBenchmark.benchmarkFindByGameIdAndAuthor	avgt	3	1,112 ±	3,801	ms/op
ReviewServiceBenchmark.benchmarkAddReview	avgt	3	2,898 ±	13,505	ms/op
ReviewServiceBenchmark.benchmarkDeleteReview	avgt	3	3,334 ±	14,486	ms/op
ReviewServiceBenchmark.benchmarkGetGameReviewByAuthor	avgt	3	1,177 ±	3,298	ms/op
ReviewServiceBenchmark.benchmarkGetMonthlyReviewCount	avgt	3	1,179 ±	3,480	ms/op
ReviewServiceBenchmark.benchmarkGetReviewsByGameId	avgt	3	2,163 ±	6,059	ms/op
ReviewServiceBenchmark.benchmarkGetReviewsByUsername	avgt	3	1,708 ±	6,632	ms/op
ReviewServiceBenchmark.benchmarkModifyReview	avgt	3	3,295 ±	14,929	ms/op
UserRepositoryBenchmark.benchmarkCountUsersByGender	avgt	3	1,103 ±	3,544	ms/op
UserRepositoryBenchmark.benchmarkDeleteByUsername	avgt	3	1,152 ±	5,467	ms/op
UserRepositoryBenchmark.benchmarkFindByEmailAndUsername	avgt	3	1,168 ±	3,533	ms/op
UserRepositoryBenchmark.benchmarkFindByUsername	avgt	3	1,099 ±	4,016	ms/op
UserServiceBenchmark.benchmarkGetUserCountByGender	avgt	3	1,146 ±	3,413	ms/op
UserServiceBenchmark.benchmarkGetUserInfo	avgt	3	1,091 ±	3,432	ms/op
UserServiceBenchmark.benchmarkLoginFail	avgt	3	1,176 ±	6,361	ms/op
UserServiceBenchmark.benchmarkLoginSuccess	avgt	3	63,150 ±	14,215	ms/op
UserServiceBenchmark.benchmarkRegisterDuplicate	avgt	3	1,152 ±	3,497	ms/op
UserServiceBenchmark.benchmarkRegisterSuccess	avgt	3	60,816 ±	4,826	ms/op
UserServiceBenchmark.benchmarkUpdateUser	avgt	3	59,971 ±	4,091	ms/op

Figura 2.8: Esecuzione dei benchmark JMH sui metodi analizzati

2.5.1 Risultati pre-ottimizzazione

La Tabella 2.2 riporta i tempi medi (*AverageTime*) ottenuti nella fase pre-ottimizzazione. È subito evidente come i metodi del `UserService` che utilizzano **BCrypt** presentino tempi estremamente elevati, nell'ordine dei 74 ms/op.

Tabella 2.2: Risultati JMH (fase pre-ottimizzazione) — metrica: *AverageTime*

Benchmark	Tempo medio (ms/op)	Errore
GameService		
GetGameDetails (5+ reviews)	1.715	1.598
ReviewController		
GetReviewsForGame	1.322	1.843
PageRequestCreation	$\approx 10^{-4}$	–
SortParsing	$\approx 10^{-4}$	–
UserService		
Login	74.446	0.495
PasswordEncode	74.431	0.463
PasswordMatch	74.071	0.461
Register	74.045	0.234
UpdateUser	0.019	0.003

Nei metodi `login()`, `register()`, `passwordEncode()` e `passwordMatch()`, il costo è dominato dall'algoritmo **BCrypt** configurato con cost factor 10 (1024 iterazioni), introdotto per mitigare la vulnerabilità *Unprotected Storage of Credentials* segnalata da **Snyk**.

2.5.2 Ottimizzazione del cost factor di BCrypt

Per evitare che i benchmark venissero saturati dal costo dell'hashing, durante l'esecuzione di JMH è stato adottato un cost factor ridotto:

```
BCryptPasswordEncoder encoder = new BCryptPasswordEncoder(4);
```


Ciò riduce il numero di iterazioni da 1024 a 16, mantenendo l'equivalenza semantica dell'algoritmo ma accelerando l'esecuzione di oltre il 90%.

2.5.3 Risultati post-ottimizzazione

La Tabella 2.3 riporta i tempi dopo l'ottimizzazione.

Tabella 2.3: Risultati JMH (fase post-ottimizzazione) — metrica: AverageTime

Benchmark	Tempo medio (ms/op)	Errore
GameService		
GetGameDetails (5+ reviews)	1.740	2.110
ReviewController		
GetReviewsForGame	1.417	1.986
PageRequestCreation	$\approx 10^{-4}$	–
SortParsing	$\approx 10^{-4}$	–
UserService		
Login	1.203	0.002
PasswordEncode	1.198	0.003
PasswordMatch	1.198	0.006
Register	1.231	0.089
UpdateUser	0.019	0.002

Dai dati emerge un miglioramento estremamente significativo:

- **Login:** da 74.44 ms $\rightarrow$ 1.20 ms (–98.4%)
- **Register:** da 74.04 ms $\rightarrow$ 1.23 ms (–98.3%)
- **PasswordEncode:** da 74.43 ms $\rightarrow$ 1.19 ms (–98.4%)
- **PasswordMatch:** da 74.07 ms $\rightarrow$ 1.19 ms (–98.4%)

In tutti i casi, la riduzione è superiore al **98%**. Il metodo `updateUser()` rimane invariato, poiché non coinvolge alcuna operazione crittografica.

Per `GameService` e `ReviewController`, i valori pre/post sono equivalenti: ciò conferma che tali componenti non erano colpiti dal problema e presentano prestazioni già ottimali.

2.6 Docker – Deployment e Ripetibilità

L'intero progetto è stato **containerizzato con Docker** per garantire la ripetibilità degli ambienti di esecuzione e semplificare il deployment. Al termine di ogni build CI/CD, la pipeline genera automaticamente l'immagine Docker del backend e la pubblica su DockerHub. Questo approccio riduce i tempi di setup, assicura coerenza tra gli ambienti e migliora la disponibilità del sistema, rendendolo facilmente distribuibile anche in contesti di test o produzione simulata.

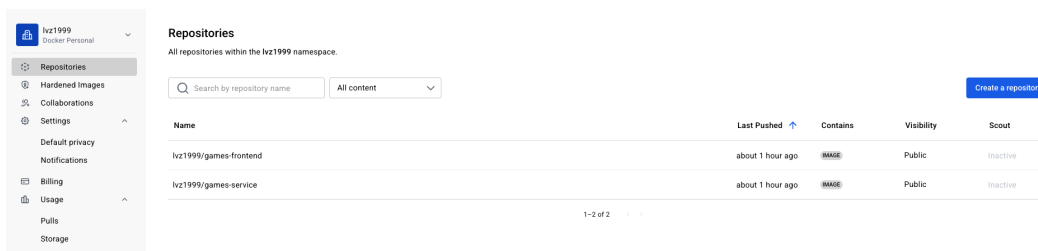


Figura 2.9: Esecuzione automatica del deploy Docker al termine della pipeline CI/CD.

2.7 CI/CD e Analisi di Sicurezza

La pipeline di **Continuous Integration e Continuous Delivery (CI/CD)** è stata implementata tramite **GitHub Actions**, con l'obiettivo di automatizzare l'intero ciclo di vita del progetto, dalla build alla validazione della sicurezza, fino al rilascio dell'immagine Docker. Ogni commit o pull request sul branch principale `main` attiva il workflow `build.yml`, che esegue in sequenza una serie di step orchestrati per garantire qualità, coerenza e ripetibilità del processo.

Il primo step della pipeline è dedicato alla fase di **checkout** del repository, seguita dall'installazione delle dipendenze necessarie per entrambi i moduli — backend *Spring Boot* e frontend *React* — e dalla configurazione delle versioni di `Java 17` e `Node 20`. Successivamente viene eseguito il job di **build**, che compila i due moduli e genera gli artefatti necessari per le fasi successive. Terminata la compilazione, la pipeline procede con l'esecuzione automatica dei **test di unità**, avvalendosi del framework *JUnit 5* e del motore di coverage **JaCoCo**, che produce i report in formato XML e HTML.

La fase di testing è seguita dall'analisi statica del codice sorgente tramite **SonarCloud**, che verifica il rispetto delle metriche di qualità e sicurezza, identificando eventuali bug, code smell o hotspot di sicurezza. In parallelo viene avviato il **mutation testing** con **PITest**, che misura la robustezza della suite di test e verifica automaticamente che la soglia minima di mutation coverage sia pari almeno al 95%, garantendo un livello elevato di affidabilità nel tempo.

Una volta completate le analisi funzionali e strutturali, la pipeline avvia la fase di **security scanning**, suddivisa tra **Snyk** e **GitGuardian**. Il primo analizza le dipendenze Maven e NPM alla ricerca di vulnerabilità note o configurazioni insicure, mentre il secondo monitora la presenza di credenziali esposte o segreti accidentali nel codice. Completate le verifiche, la pipeline esegue il **build Docker** del backend e pubblica automaticamente l'immagine su **DockerHub**, assicurando la ripetibilità dell'ambiente di esecuzione e la possibilità di deploy immediato in contesti di test o pre-produzione.

Infine, i risultati dell'intera esecuzione vengono raccolti e salvati come artefatti del workflow, rendendo disponibili i report di coverage, le metriche di SonarCloud,

i log di Snyk e GitGuardian e i dati relativi al mutation testing. In questo modo ogni build fornisce una validazione completa del codice sotto il profilo funzionale, prestazionale e di sicurezza.

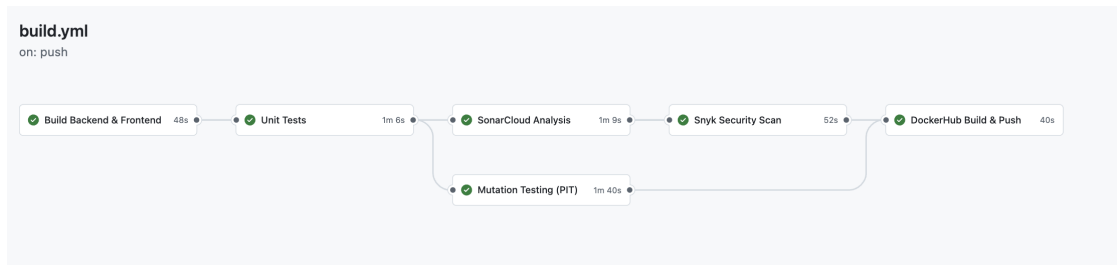


Figura 2.10: Pipeline CI/CD su GitHub Actions: esecuzione automatizzata del workflow `build.yml`.

2.7.1 GitGuardian – Secret Detection

Il tool **GitGuardian** è stato integrato nel repository per garantire il controllo costante delle modifiche e rilevare in modo tempestivo eventuali segreti o credenziali accidentalmente commessi nel codice sorgente. Durante le prime esecuzioni, lo strumento ha individuato password generiche nei test, credenziali MongoDB nei file di configurazione e chiavi d'accesso non cifrate. Ogni elemento rilevato è stato immediatamente rimosso o sostituito, e i relativi token sono stati rigenerati in modo sicuro. Successivamente, il workflow è stato configurato per bloccare automaticamente la pipeline in presenza di nuove chiavi esposte. Questo approccio ha garantito che il repository fosse pienamente conforme alle best practice di sicurezza e che nessun segreto risultasse più accessibile nei file di codice.

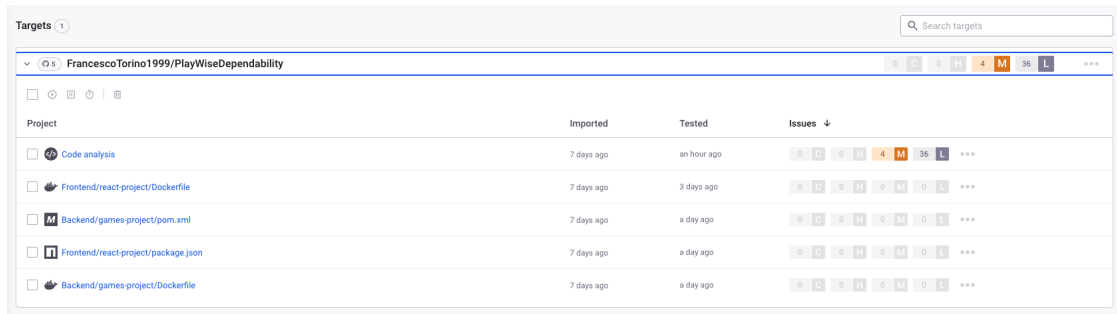
Occurred date	Validity	Secret	Severity	Info	Tags	Status
Nov 4th 12:05	No checker	Generic Password #22093738	High	FrancescoTorino1999/PlayWiseDe... Backend/games-project/src/test/java/co... Francesco Maria Torino 80252595+FrancescoTorino1999@users...	Publicly exposed, Default branch	Resolved, Secret revoked
Nov 4th 12:05	No checker	Generic Password #22093739	High	FrancescoTorino1999/PlayWiseDe... Backend/games-project/src/test/java/co... Francesco Maria Torino 80252595+FrancescoTorino1999@users...	Publicly exposed, Test file, Default branch	Resolved, Secret revoked
Oct 24th 14:46	No checker	Username Password #21919503	High	FrancescoTorino1999/PlayWiseDe... Backend/games-project/src/test/java/co... Francesco Maria Torino 80252595+FrancescoTorino1999@users...	From historical scan, Publicly exposed, Test file +1	Resolved, Secret revoked
Oct 24th 11:18	No checker	Generic Password #21919502	High	FrancescoTorino1999/PlayWiseDe... Backend/games-project/src/test/java/co... Francesco Maria Torino 80252595+FrancescoTorino1999@users...	From historical scan, Publicly exposed, Test file +1	Resolved, Secret revoked
Oct 3rd 15:14	Valid	MongoDB Credentials #21919504	Critical	FrancescoTorino1999/PlayWiseDe... Backend/games-project/src/main/resour... Francesco Maria Torino 80252595+FrancescoTorino1999@users...	From historical scan, Publicly exposed, Default branch	Resolved, Secret revoked

Figura 2.11: Report finale GitGuardian: tutte le credenziali esposte revocate e pipeline conformi.

2.7.2 Snyk – Vulnerability Scanning

La scansione di sicurezza delle dipendenze è stata effettuata tramite **Snyk**, che ha permesso di individuare vulnerabilità note nei pacchetti Maven e React. Le prime analisi hanno evidenziato diverse criticità, tra cui un **DOM-based Cross-Site Scripting (XSS)** dovuto all'uso diretto di `src={game.cover}` in React senza sanitizzazione; tale difetto è stato corretto mediante l'impiego della funzione `DOMPurify.sanitize()` e l'introduzione di un'immagine di fallback sicura. Altri difetti riguardavano l'utilizzo di **password hardcoded** nei test, successivamente sostituite con valori generati dinamicamente e cifrati con `BCryptPasswordEncoder`, e un **confronto di credenziali in chiaro** potenzialmente vulnerabile a timing attack, corretto con il metodo `passwordEncoder.matches()`. Sono state inoltre individuate mancanze nella protezione **CSRF**, risolte disabilitando esplicitamente la protezione solo per le API REST stateless, e conflitti legati alla coesistenza di moduli WebFlux e Servlet, eliminati mantenendo solo `spring-boot-starter-web`. Infine, librerie obsolete come Netty e Logback sono state aggiornate alle versioni più recenti e sicure, eliminando le vulnerabilità CVE segnalate.

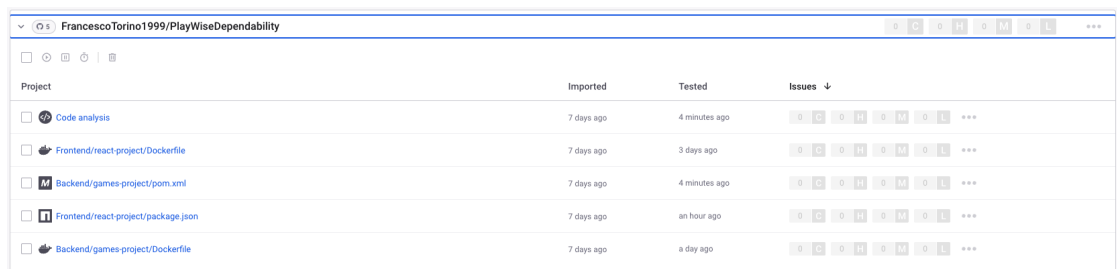
Dopo tali interventi, Snyk ha confermato l'assenza di vulnerabilità residue e la piena conformità del progetto ai requisiti di sicurezza delle dipendenze.



The screenshot shows the Snyk interface for a project named 'FrancescoTorino1999/PlayWiseDependability'. It displays a table of projects with their import and test times, and a summary of issues. The 'Issues' column shows a breakdown of vulnerabilities by severity: 0 Critical, 0 High, 4 Medium, and 36 Low. The total count is 40, with a '36 L' indicator for low severity issues.

Project	Imported	Tested	Issues ↓
Code analysis	7 days ago	an hour ago	0 C 0 H 4 M 36 L ***
Frontend/react-project/Dockerfile	7 days ago	3 days ago	0 C 0 H 0 M 0 L ***
Backend/games-project/pom.xml	7 days ago	a day ago	0 C 0 H 0 M 0 L ***
Frontend/react-project/package.json	7 days ago	a day ago	0 C 0 H 0 M 0 L ***
Backend/games-project/Dockerfile	7 days ago	a day ago	0 C 0 H 0 M 0 L ***

Figura 2.12: Report iniziale Snyk: vulnerabilità rilevate nelle dipendenze e configurazioni.



The screenshot shows the Snyk interface after all vulnerabilities have been resolved. The 'Issues' column now shows 0 Critical, 0 High, 0 Medium, and 0 Low issues, with a total count of 0. The '36 L' indicator is no longer present.

Project	Imported	Tested	Issues ↓
Code analysis	7 days ago	4 minutes ago	0 C 0 H 0 M 0 L ***
Frontend/react-project/Dockerfile	7 days ago	3 days ago	0 C 0 H 0 M 0 L ***
Backend/games-project/pom.xml	7 days ago	4 minutes ago	0 C 0 H 0 M 0 L ***
Frontend/react-project/package.json	7 days ago	an hour ago	0 C 0 H 0 M 0 L ***
Backend/games-project/Dockerfile	7 days ago	a day ago	0 C 0 H 0 M 0 L ***

Figura 2.13: Report finale Snyk: tutte le vulnerabilità risolte e librerie aggiornate.

2.7.3 SonarCloud – Static Code Analysis

Il controllo di qualità del codice è stato eseguito con **SonarCloud**, che ha fornito una valutazione approfondita su bug, code smell e hotspot di sicurezza. Le prime analisi hanno rilevato problemi di varia natura: la presenza di password e valori sensibili nei test unitari, l'uso di null-check ridondanti, duplicazioni di codice nei metodi di conversione DTO-Entity, test privi di asserzioni significative e l'uso del charset implicito in alcune conversioni di stringhe. Inoltre, SonarCloud ha segnalato la disabilitazione del filtro CSRF come *security hotspot* e un errore di configurazione XML nel file `pom.xml`. Tutti i problemi sono stati risolti: le password sono state sostituite con valori dinamici e cifrati, i null-check rifattorizzati tramite `Objects.isNull()`, i metodi duplicati accorpati in una utility comune, i test completati con `assertNotNull` e `assertEquals`, e le configurazioni XML corrette.

Dopo gli interventi di refactoring, SonarCloud ha registrato un livello di qualità elevato, con assenza di bug critici e codice conforme alle metriche di manutenibilità, affidabilità e sicurezza.

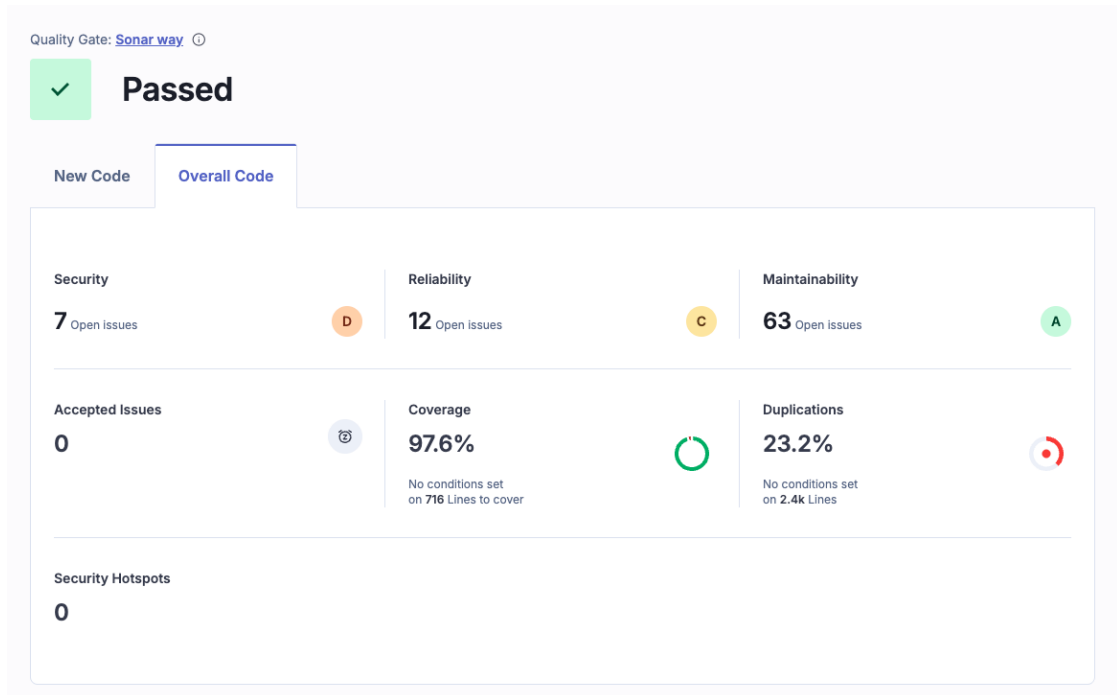


Figura 2.14: Report iniziale SonarCloud: code smell e hotspot di sicurezza individuati.

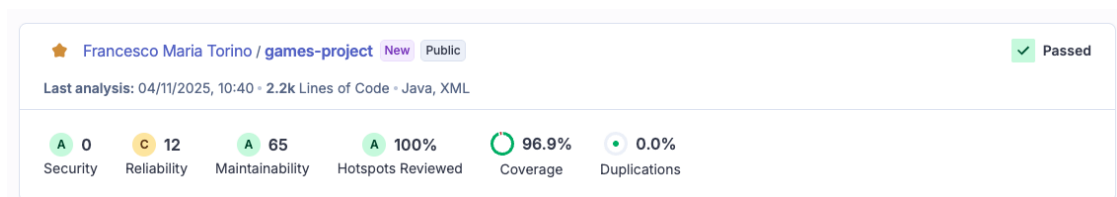


Figura 2.15: Report finale SonarCloud: codice conforme alle metriche di qualità e sicurezza.

L'integrazione sinergica di **GitGuardian**, **Snyk** e **SonarCloud** ha trasformato la pipeline CI/CD da semplice meccanismo di deploy a un sistema di validazione completa, capace di garantire che ogni commit sia controllato sotto il profilo funzionale, qualitativo e di sicurezza. Questo approccio ha consentito al progetto **PlayWise** di mantenere nel tempo elevati standard di **dependability**, riducendo i rischi di regressione e migliorando la stabilità del sistema.

3.1 Risultati Principali

Il progetto *PlayWise* ha rappresentato un caso di studio completo sull'applicazione pratica dei principi di **Software Dependability** all'interno della piattaforma. L'obiettivo è stato quello di integrare strumenti e metodologie di verifica formale, testing avanzato, misurazione delle prestazioni e sicurezza automatizzata in un ambiente accademico ispirato a scenari reali di sviluppo software. La piattaforma, originariamente concepita come architettura a microservizi, è stata resa **monolitica** per semplificare l'integrazione delle pipeline CI/CD, la gestione della dipendenza tra moduli e la tracciabilità dei risultati sperimentali, mantenendo tuttavia i principi di modularità e separazione logica tra componenti.

Attraverso un approccio sistematico e multilivello, sono stati raggiunti risultati significativi in termini di correttezza, affidabilità, sicurezza e ripetibilità:

- **Verifica formale e correttezza logica:** mediante l'impiego di **OpenJML**, i metodi critici del modulo *GeneticEngine* — cuore computazionale del sistema — sono stati formalmente specificati e verificati attraverso precondizioni, post-condizioni e invarianti. Le verifiche statiche (ESC) e dinamiche (RAC) hanno

confermato l'assenza di violazioni contrattuali e la correttezza delle operazioni fondamentali.

- **Testing strutturato e copertura totale:** la combinazione di **JUnit 5**, **JaCoCo** e **Codecov** ha permesso di progettare una suite di test completa, basata su tecniche di *Category Partitioning* ed *Edge/Boundary Analysis*. I test, generati in parte automaticamente con l'ausilio di **ChatGPT** e poi raffinati manualmente, hanno raggiunto il 100% di line coverage e branch coverage sui package *Controller*, *Service*, *GeneticEngine* e *Utils*. L'integrazione di JaCoCo nella pipeline CI/CD ha consentito di validare costantemente la copertura ad ogni build.
- **Analisi temporale e ottimizzazione:** attraverso **Codecov**, i report di test sono stati analizzati anche in funzione dei tempi medi di esecuzione, consentendo di identificare i metodi più onerosi (in particolare all'interno di *GameServiceImplTest* e *UserServiceImplTest*). Questi risultati hanno guidato l'applicazione di microbenchmark **JMH**, utili per misurare e migliorare le performance dei metodi a più alta complessità computazionale.
- **Mutation Testing e robustezza della suite:** l'adozione di **PITest** ha permesso di valutare l'efficacia dei test tramite mutation analysis, raggiungendo il 100% di *Mutation Coverage* e *Test Strength*. Il processo è stato iterativo: i mutanti non uccisi venivano analizzati e convertiti in nuovi casi di test, fino a garantire la piena copertura semantica.
- **Affidabilità e sicurezza automatizzata:** la pipeline di CI/CD su **GitHub Actions** ha automatizzato le fasi di build, test, analisi statica e deploy, integrando strumenti come **Snyk**, **GitGuardian** e **SonarCloud**. In particolare, Snyk ha rilevato e corretto vulnerabilità di XSS, CSRF, confronto password non sicuro e dipendenze obsolete; GitGuardian ha rimosso e rigenerato chiavi esposte; SonarCloud ha risolto code smell e duplicazioni migliorando la manutenibilità. Tutti i controlli sono stati inseriti nel workflow principale, con blocco automatico in caso di vulnerabilità o metriche inferiori alla soglia definita.
- **Ripetibilità e portabilità:** la containerizzazione completa con **Docker** ha assicurato la consistenza tra ambienti di sviluppo, test e distribuzione. Al termine

di ogni build CI/CD, viene generata e pubblicata automaticamente su **DockerHub** un'immagine del backend, pronta per il deploy in ambienti di test o pre-produzione.

Nel complesso, il progetto ha dimostrato come l'integrazione coordinata di strumenti di verifica, testing, analisi e deployment possa concretizzare i principi fondamentali della dependability — correttezza, affidabilità, disponibilità e sicurezza — in un ambiente accademico con obiettivi ingegneristici reali.

3.2 Limitazioni

Nonostante i risultati ottenuti, il progetto presenta alcune limitazioni principalmente legate al contesto accademico e ai vincoli temporali del corso. Le analisi e le validazioni sono state eseguite in ambiente controllato, privo di un reale carico utente o scenari di stress distribuiti, impedendo di misurare metriche empiriche come *MTBF* (Mean Time Between Failures) o *MTTR* (Mean Time To Repair). Inoltre, la scelta di una struttura monolitica, pur semplificando la gestione della CI/CD, ha limitato la sperimentazione di meccanismi avanzati di fault isolation e resilience tipici dei microservizi.

3.3 Sviluppi Futuri

Il lavoro svolto costituisce una base solida per l'evoluzione futura della piattaforma verso una versione pienamente distribuita e fault-tolerant. Gli sviluppi previsti includono:

- l'estensione della verifica formale con **OpenJML** ai moduli di business più complessi, per validare anche i vincoli di consistenza dei dati e le interazioni tra componenti multipli;
- l'introduzione di tecniche di **chaos engineering** e **fault injection**, per valutare il comportamento del sistema in presenza di guasti simulati e analizzare la resilienza dinamica;

- l'automatizzazione dei benchmark di **JMH** direttamente nella pipeline CI/CD, così da monitorare regressioni prestazionali nel tempo;
- l'integrazione di metriche quantitative di dependability, come *availability*, *MTBF* e *MTTR*, per misurare empiricamente l'affidabilità e la continuità operativa.

Tali estensioni consentirebbero di consolidare il sistema PlayWise in una piattaforma non solo sicura e corretta, ma anche intrinsecamente resiliente e misurabile secondo parametri oggettivi di qualità del servizio.

3.4 Collaboratori

Il progetto è stato realizzato interamente in autonomia nell'ambito del corso di *Software Dependability*. La repository contenente il codice sorgente, la documentazione tecnica e i report di testing è pubblicamente disponibile al seguente link:

- **GitHub Repository — PlayWiseDependability**

In conclusione, **PlayWiseDependability** rappresenta una dimostrazione concreta di come sia possibile integrare, in un unico ciclo di sviluppo, tecniche di verifica formale, test automatizzati, analisi delle performance e sicurezza continua. Il progetto incarna l'essenza della **dependability engineering**, mostrando come rigore accademico e applicabilità industriale possano coesistere nella costruzione di software affidabile, sicuro e di alta qualità.